高级语言程序设计

实验报告

南开大学 计算机大类

姓名：唐健

学号：2412103

班级：模拟 2-1 班

2025 年5 月 11 日

目录

一、 作业题目

利用 easyx 工具实现对《空洞骑士：丝之歌》的一场 boss 战模拟

二、 开发软件

Visual Studio 2022（利用 easyx 图形库）

三、 课题要求

运用C++语言进行程序开发。

基于面向对象的编程思想进行代码设计。

借助 easyx 图形库实现图形化的游戏界面与交互效果

四、 主要流程

4.1 文件关系分析及主函数结构分析

4.1.1 Actor.h/cpp

作用：该文件主要定义了 Actor 类，此为游戏中角色的基础类。它包含了角色的各种属性和方法声明，这些属性和方法用于描述角色的状态和行为。

具体内容：

属性： 包含重力 gravity、加速度 acceleration、速度 speed、攻击方向 attackdir、位置 position、生命值 HP 等。还定义了角色是否可受伤的标志 CanBeHurt、无敌时间 UnHurtableTime 以及角色的朝向 Facing_Right。

方法： 声明了消息处理函数 messagedealer() 和构造函数 Actor() 以及 Actor(double nx, double ny)。

```
#pragma once
#include<Windows.h>
#include<time.h>
#include<mmsystem.h>
#include "Vector2.h"
#include "Point.h"

//物体类
class Actor
{
public:
    double gravity = 2.5;
    double acceleration = 0;
    Vector2 speed;
    Vector2 attackdir;
    Point position;
    double HP = 3;

    bool CanBeHurt = 1;
    int UnHurtableTime = 10;
    bool Facing_Right = 1;

    int messagedealer();
    Actor();
    Actor(double nx, double ny);
};
```

4.1.2 AnimationA.h/cpp

作用：定义了 AnimationA 类，这是一个动画的基础类，主要用于存储和管理动画的基本信息。

具体内容：

属性：包括动画的最大宽度 maxwidth 和最大高度 maxheight、帧列表 frame_list、帧数 frame_num、冷却时间 CD 和 TwiceCD，以及移动数据 dmove、帧间隔 frameinterval 和临时帧间隔 tmp_interval。

方法：声明了加载动画的函数 LoadAnimationA(LPCTSTR path, int frame_num)。

```
#include "AnimationA.h"
void AnimationA::LoadAnimationA(LPCTSTR path, int frame_number)
{
    dmove.resize(frame_number); tmp_interval.resize(frame_number);
    frameinterval.resize(frame_number);
    for (int i = 0; i < frame_number; i++)
    {
        dmove[i].resize(2);
    }
    frame_num = frame_number;
    TCHAR path_file[256];
    for (int i = 0; i < frame_num; i++)
    {
        _stprintf_s(path_file, path, i);
        IMAGE* frame = new IMAGE;
        loadimage(frame, path_file);
        if (maxwidth < frame->getwidth())
            maxwidth = frame->getwidth();
        if (maxheight < frame->getheight())
            maxheight = frame->getheight();
        frame_list.push_back(frame);
    }
}
```

4.1.3 Animation.h/.cpp

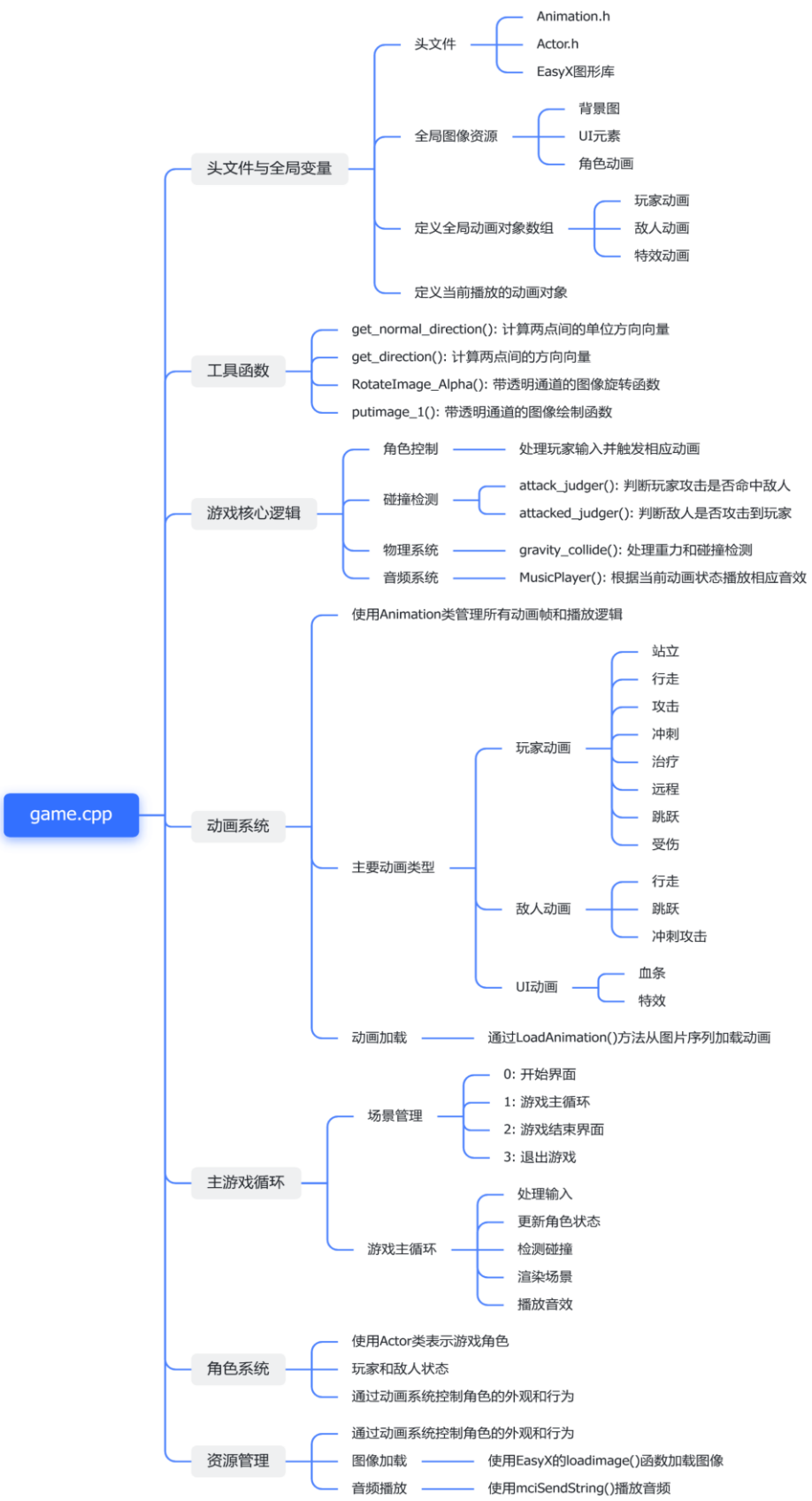
作用：继承了 Animation 类，实现了 Animation 类的部分方法，特别是加载击中特效的方法。

具体内容：

方法：实现了 LoadHitEffect 方法，该方法用于从指定路径加载击中特效的图像，并将其存储在 HitEffects 向量中，同时记录特效的起始帧和结束帧

```cpp
#include "Animation.h"


Animation::Animation(const Animation& anim)
{
    maxwidth = anim.maxwidth; maxheight = anim.maxheight; CD = anim.CD;
    frame_num = anim.frame_num;
    animationname = anim.animationname;
    point = anim.point; current_frameidx = anim.current_frameidx;
    loop_index = anim.loop_index; canloop = anim.canloop;
    dmove = anim.dmove; frameinterval = anim.frameinterval;
    Erelative = anim.Erelative; CD = anim.CD; relative = anim.relative;
    frame_list.assign(anim.frame_list.begin(), anim.frame_list.end());
}


void Animation::LoadAnimation(LPCTSTR path, int frame_num, AnimationName name)
{
    AnimationA::LoadAnimationA(path, frame_num);
    animationname = name; relative.resize(frame_num);
    for (int i = 0; i < frame_num; i++)
    {
        relative[i].vx = 0; relative[i].vy = 0;
    }
    //times = time;
}


void Animation:: LoadEffect(LPCTSTR path, int start, int end, int num) {
    Effects.resize(0); Erelative.resize(num);
    TCHAR path_file[256];
    for (int i = 0; i < num; i++)
    {
        _stprintf_s(path_file, path, i);
        IMAGE* img = new IMAGE; loadimage(img, path_file);
        Effects.push_back(img);
    }
    StartFrame = start; EndFrame = end;
}


void Animation:: LoadHitEffect(LPCTSTR path, int start, int end, int num) {
    HitEffects.resize(0); HitErelative.resize(num);
    TCHAR path_file[256];
    for (int i = 0; i < num; i++)
    {
        _stprintf_s(path_file, path, i);
        IMAGE* img = new IMAGE; loadimage(img, path_file);
        HitEffects.push_back(img);
    }
     HitStartFrame = start; HitEndFrame = end;
}
```

4.1.4 Game.cpp


game.cpp
头文件与全局变量
头文件
Animation.h
Actor.h
EasyX图形库
全局图像资源
背景图
UI元素
角色动画
定义全局动画对象数组
玩家动画
敌人动画
特效动画
定义当前播放的动画对象
工具函数
get_normal_direction(): 计算两点间的单位方向向量
get_direction(): 计算两点间的方向向量
RotateImage_Alpha(): 带透明通道的图像旋转函数
putimage_1(): 带透明通道的图像绘制函数
游戏核心逻辑
角色控制
处理玩家输入并触发相应动画
碰撞检测
attack_judger(): 判断玩家攻击是否命中敌人
attacked_judger(): 判断敌人是否攻击到玩家
物理系统
gravity_collide(): 处理重力和碰撞检测
音频系统
MusicPlayer(): 根据当前动画状态播放相应音效
动画系统
使用Animation类管理所有动画帧和播放逻辑
主要动画类型
玩家动画
站立
行走
攻击
冲刺
治疗
远程
跳跃
受伤
敌人动画
行走
跳跃
冲刺攻击
UI动画
血条
特效
动画加载
通过LoadAnimation()方法从图片序列加载动画
主游戏循环
场景管理
0: 开始界面
1: 游戏主循环
2: 游戏结束界面
3: 退出游戏
游戏主循环
处理输入
更新角色状态
检测碰撞
渲染场景
播放音效
角色系统
使用Actor类表示游戏角色
玩家和敌人状态
通过动画系统控制角色的外观和行为
资源管理
通过动画系统控制角色的外观和行为
图像加载
使用EasyX的loadimage()函数加载图像
音频播放
使用mciSendString()播放音频

4.2 游戏控制与动作实现

按键控制：

游戏采用常见的键盘控制方式，在 actor.h 中定义 messagedealer 函数来处理玩家的键盘输入

其中 W、A、S、D 用于角色移动，J 进行攻击，K 实现跳跃，Space 用于冲刺，I 释放远程技能，H 触发超级冲刺，L 执行回血操作

动作时长控制：

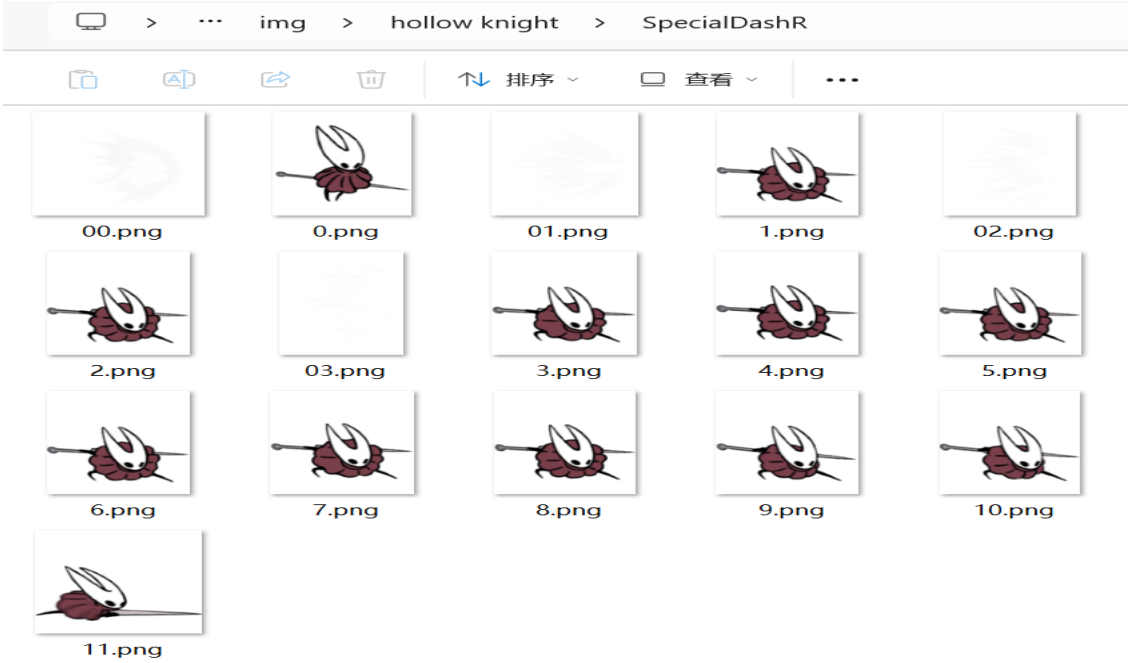
通过维护一个 vector 数组来控制每个动作的时长，即帧间隔。以治疗动作 PLAYER_CURE 为例，利用 Animation 类中的 LoadAnimation 函数加载图片素材。该动作由 45 帧构成，特效部分从第 20 帧到 38 帧共 18 帧。通过循环初始化 frameinterval 数组，默认所有帧间隔为 2

```
PLAYER_CURE[0].LoadAnimation(_T("img/hollow knight/Cure/%d.png"), 45, PLAYER_CURE0);
PLAYER_CURE[1].LoadAnimation(_T("img/hollow knight/CureR/%d.png"), 45, PLAYER_CURE0);
PLAYER_CURE[0].LoadEffect(_T("img/hollow knight/Cure/Effects/0%d.png"), 20, 38, 18);
PLAYER_CURE[1].LoadEffect(_T("img/hollow knight/CureR/Effects/0%d.png"), 20, 38, 18);
for (int i = 0; i < 45; i++)
{
    PLAYER_CURE[1].frameinterval[i] = 2; PLAYER_CURE[0].frameinterval[i] = 2;
    if (i >= 13 && i <= 17) { PLAYER_CURE[1].frameinterval[i] = 1;PLAYER_CURE[0].frameinterval[i] = 1; }
}PLAYER_CURE[1].CD = 30; PLAYER_CURE[1].tmp_interval = PLAYER_CURE[1].frameinterval;
PLAYER_CURE[0].tmp_interval = PLAYER_CURE[1].frameinterval;PLAYER_CURE[0].frameinterval = PLAYER_CURE[1].frameinterval;

for (int i = 0; i < 18; i++)
{
    PLAYER_CURE[0].Erelative[i].vx = -80;PLAYER_CURE[0].Erelative[i].vy = -50;
    PLAYER_CURE[1].Erelative[i].vx = -80; PLAYER_CURE[1].Erelative[i].vy = -50;
}
PLAYER_CURE[0].canloop = false;  // 不循环播放
PLAYER_CURE[1].canloop = false;
PLAYER_CURE[0].CD = 60;
PLAYER_CURE[1].CD = 60;
```

同时对 13 到 17 帧进行特殊处理，将帧间隔缩短为 1，从而加快这部分动画的播放速度。

特殊动作实现：在实现特殊动作如 specialdash（蓄力冲刺）时，通过

复制相同的素材（如 3 到 8 帧），巧妙地实现了所需效果。尽管并非通过复杂的代码逻辑实现，但有效满足了需求，使游戏能够正常运行。



（三）技能实现 - 黄蜂女投掷针技能

动作定义与加载：在 animationname 头文件中，枚举 REMOTE0 和 NEEDLE0 两个动作名称。在 game 主文件中分别设置 remote 和 needle 两个动作，并进行加载。

```
PLAYER_REMOTE[0].LoadAnimation(_T("img/hollow knight/RemoteSkill/%d.png"), 27, PLAYER_REMOTE0);
PLAYER_REMOTE[1].LoadAnimation(_T("img/hollow knight/RemoteSkillR/%d.png"), 27, PLAYER_REMOTE0);
PLAYER_REMOTE[0].LoadEffect(_T("img/hollow knight/RemoteSkill/0%d.png"), 0, 15, 16);
PLAYER_REMOTE[1].LoadEffect(_T("img/hollow knight/RemoteSkillR/0%d.png"), 0, 15, 16);
for (int i = 0; i < 27; i++)
{
    PLAYER_REMOTE[1].frameinterval[i] = 2;PLAYER_REMOTE[0].frameinterval[i] = 2;
    if (i >= 13 && i <= 17) { PLAYER_REMOTE[1].frameinterval[i] = 1;PLAYER_REMOTE[0].frameinterval[i] = 1; }
}PLAYER_REMOTE[1].CD = 30; PLAYER_REMOTE[1].tmp_interval = PLAYER_REMOTE[1].frameinterval;
PLAYER_REMOTE[0].tmp_interval = PLAYER_REMOTE[1].frameinterval;PLAYER_REMOTE[0].frameinterval = PLAYER_REMOTE[1].framein

for (int i = 0; i < 15; i++)
{
    PLAYER_REMOTE[0].Erelative[i].vx = -80;PLAYER_REMOTE[0].Erelative[i].vy = -50;
    PLAYER_REMOTE[1].Erelative[i].vx = -80;PLAYER_REMOTE[1].Erelative[i].vy = -50;
}
PLAYER_REMOTE[0].canloop = false;
PLAYER_REMOTE[1].canloop = false;
PLAYER_REMOTE[0].CD = 60;
PLAYER_REMOTE[1].CD = 60;
```

针的状态管理：使用多个变量来记录针的飞行方向、飞行速度、激活状态等信息。在游戏循环中，不断更新针的位置和动画帧。

```
Vector2 needle_position;
bool needle_active = false;
int needle_direction = 0; // 针的飞行方向(0:左,1:右)
int needle_speed = 15; // 针的飞行速度
int needle_frame = 0; // 针的动画帧
int needle_frame_counter = 0; // 针的帧计数



NEEDLE[0].LoadAnimation(_T("img/hollow knight/RemoteSkill/needle/%d.png"), 7, NEEDLE0);
NEEDLE[1].LoadAnimation(_T("img/hollow knight/RemoteSkill/needleR/%d.png"), 7, NEEDLE0);
//// 设置针动画的帧间隔
for (int i = 0; i < 7; i++) {
    NEEDLE[0].frameinterval[i] = 2;
    NEEDLE[1].frameinterval[i] = 2;

    // 初始化临时帧间隔
    NEEDLE[0].tmp_interval[i] = NEEDLE[0].frameinterval[i];
    NEEDLE[1].tmp_interval[i] = NEEDLE[1].frameinterval[i];
}

NEEDLE[0].canloop = false;  // 不循环播放
NEEDLE[1].canloop = false;
NEEDLE[0].CD = 0;  // 冷却时间
NEEDLE[1].CD = 0;
```

碰撞处理：在 attack_judger 函数中，定义玩家攻击效果和敌人位置的

中心点，计算攻击效果和敌人中心的水平（dx）垂直（dy）距离。若水平垂直距离和分别小于两者宽高和的一半，则判定为命中。为确保碰撞计算准确，按照特效的第一帧来计算玩家攻击效果的中心点，并调整了 needle 的第一帧动画尺寸。

```
bool attack_judger(Actor& player, Actor& enemy)
{
    if (enemy.UnHurtableTime == 0)
    {
        AnimationName Playeranimname = CurrentPlayerAnimation.animationname;
        if (CurrentEnemyAnimation.animationname == NU)
        {
            return false;
        }
        if (Playeranimname == PLAYER_ATTACK0 || Playeranimname == PLAYER_UPATTACK0)
        {
            Vector2 RelativeVector = get_direction(player.position, enemy.position);
            Point EffectCenter, EnemyCenter;
            int w1 = CurrentPlayerAnimation.Effects[0]->getwidth();
            int h1 = CurrentPlayerAnimation.Effects[0]->getheight();
            int w2 = CurrentEnemyAnimation.frame_list[CurrentEnemyAnimation.current_frameidx]->getwidth() - 50;
            int h2 = CurrentEnemyAnimation.frame_list[CurrentEnemyAnimation.current_frameidx]->getheight() - 50;
            EffectCenter.x = player.position.x + CurrentPlayerAnimation.dmove[0][0] + (2 * player.Facing_Right - 1) * w1 /
            EffectCenter.y = player.position.y + CurrentPlayerAnimation.dmove[0][1] + (2 * player.Facing_Right - 1) * h1 /
            EnemyCenter.x = enemy.position.x + w2 / 2;
            EnemyCenter.y = enemy.position.y + h2 / 2;
            if ((player.Facing_Right * 2 - 1) * RelativeVector.vx >= 0)
            {
                int dx = abs(EnemyCenter.x - EffectCenter.x);
                int dy = abs(EnemyCenter.y - EffectCenter.y);
                if (dx < (w1 + w2) / 2 &&
                    dy < (h1 + h2) / 2 && enemy.UnHurtableTime == 0)
                {
                    cout << "ok!" << endl;
                    return true;
                }
                return false;
            }
```

（四）游戏细节实现

UI 设计 - 血条：角色血条素材从网上获取并进行抠图处理，以体现与原作小骑士的区别。血条具有动画效果，掉血特效则是利用小骑士攻击特效通过 PS 修改尺寸得到。遗憾的是，本次未设计灵魂系统，

素材获取：项目中使用的图片素材数量众多，总计好几百张，基本都

需要进行抠图处理。素材来源主要是网上的相关网站，会将资源网站链接放在评论区，方便感兴趣的人使用。相关音频文件大多通过 assetstudio 解包《空洞骑士》获取，由于黄蜂骑士的语音素材较少，借用了奎若的一些语音，最终呈现出了有趣的效果。

项目打包：为了方便游玩，将项目打包成了 exe 文件，提高了游戏的可执行性和便捷性。

五、 收获与展望

（一）收获

1. 通过本次大作业，深入理解了游戏开发中图形化界面的实现原理，掌握了如何利用easyx图形库加载图片、播放动画以及实现用户交互。

2. 对面向对象编程的理解更加深刻，学会了如何封装代码，将不同的功能模块抽象成类，提高了代码的可读性、可维护性和可扩展性。

3. 了解了游戏开发中资源管理的重要性，包括图片素材、音频文件的获取与处理，以及如何优化资源以提升游戏性能。

4. 体会到实际项目开发中解决问题的方法和思路，如在实现特殊动作和技能时，通过创新的方式满足需求，不拘泥于传统的代码实现方式。

（二）展望

1. 后续计划继续完善该游戏，利用其他更专业的游戏开发引擎或工具，实现运动相机效果，使游戏视角更加灵活，增强游戏的沉浸感。

2. 尝试实现场景切换功能，丰富游戏的场景内容，提升游戏的可玩性。

3. 探索粒子动画的实现方法，为游戏添加更绚丽的特效，如攻击特效、环境特效等，提升游戏的视觉效果。